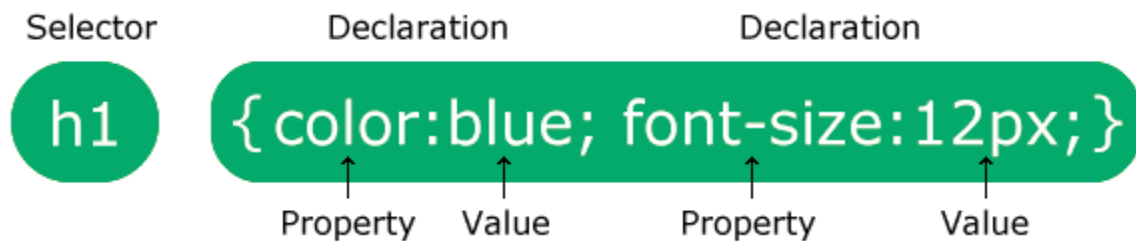


What is CSS?

- **CSS (Cascading Style Sheets)** is a stylesheet language used to define the presentation and design of a web page.
- It controls the visual aspects of web pages written in HTML (HyperText Markup Language) or XML (Extensible Markup Language), including the layout, colors, fonts, spacing, positioning, and overall look and feel.
- While **HTML** provides the structure and content of a web page (such as headings, paragraphs, images, and links), **CSS** focuses on styling and making that content visually appealing, responsive, and consistent across different devices and screen sizes.
- **Cascading** refers to the way styles are applied to elements. If multiple CSS rules conflict, the browser follows a "cascading" hierarchy to determine which style is applied based on specificity, source order, and importance.
- **Style Sheets** are documents that contain a set of rules for styling HTML elements, including properties like color, font, margins, padding, etc.

CSS Syntax

A CSS rule consists of a selector and a declaration block:



- ✓ The selector points to the HTML element you want to style.
- ✓ The declaration block contains one or more declarations separated by semicolons.
- ✓ Each declaration includes a CSS property name and a value, separated by a colon.
- ✓ Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly brace

Example: `h1 {color: blue; font-size: 24px;}`

- This rule styles all `<h1>` elements by setting the text color to **blue** and the font size to **24px**.

Importance of CSS:

1. **Separation of Content and Presentation:** CSS separates the structure of a webpage (HTML) from its design (styling). This makes the code cleaner, easier to maintain, and allows for easier updates to the design without affecting the content.
2. **Consistency Across Multiple Pages:** With CSS, the same style can be applied to multiple pages, ensuring a consistent look across the entire website. A single CSS file can control the appearance of all pages linked to it.
3. **Improved User Experience (UX):** CSS allows for visually appealing, user-friendly designs, enhancing navigation and readability, which improves the overall user experience on websites.

4. **Faster Load Times:** Using external CSS files allows browsers to cache the stylesheet, reducing the need for repeated downloads, which leads to faster loading times for web pages.
5. **Responsive Design:** CSS makes it easier to create **responsive designs** that adjust automatically to different screen sizes (mobile, tablet, desktop), ensuring that websites are accessible and user-friendly on all devices.
6. **Advanced Styling:** CSS enables advanced styling features like animations, transitions, and grid layouts, allowing developers to create dynamic and interactive web pages.
7. **Better SEO:** By separating content (HTML) from style (CSS), search engines can better index the content of a webpage, potentially improving search engine rankings.

Types of CSS

CSS (Cascading Style Sheets) can be applied in three distinct ways to style HTML documents. These are:

1. Inline CSS

- Inline CSS is a method of applying Cascading Style Sheets (CSS) directly within the HTML document, using the style attribute.
- This technique enables the developer to define CSS rules that apply only to a specific HTML element.
- Unlike other methods of applying CSS, such as external or internal CSS, inline CSS is applied on a case-by-case basis, directly within the element's tag.

Syntax

```
<tagname style="property: value;">
```

```
<!-- Content goes here -->
```

```
</tagname>
```

Example 1:

```
<p style="color: green; font-size: 18px;">This paragraph is green and 18px in font size.</p>
```

Example 2:

```
<button style="background-color: blue; color: white; padding: 10px 20px;">Click Me</button>
```

Example 3: ``

Advantages

1. **Quick Implementation:** Ideal for temporary changes or one-off styling.
2. **High Specificity:** Inline styles override other styles (external or internal).
3. **Localized:** Affects only the targeted element.

Disadvantages

1. **Limited Reusability:** Styles must be repeated for each element.
2. **Hard to Maintain:** Difficult to manage when the site grows.
3. **Messy Code:** Combines HTML structure with CSS, violating the separation of concerns.

When to Use

1. Quick, temporary styling.
2. Testing or prototyping.
3. Overriding specific element styles.

2. **Internal CSS (Embedded CSS)**

- Internal CSS, also referred to as Embedded CSS, is a method of styling HTML documents by including CSS rules within the `<style>` tag.
- This tag is placed in the `<head>` section of the HTML document.
- The styles defined within the `<style>` tag apply only to the HTML elements of the specific page in which the CSS is embedded.
- This approach allows for centralized styling within a single document, making it a suitable option for smaller websites or single-page applications.

Syntax

```
<!DOCTYPE html>
<html>
<head>
  <style>
    /* CSS rules go here */
    selector {
      property: value;
    }
  </style>
</head>
<body>
  <!-- HTML content goes here -->
</body>
</html>
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internal CSS Example</title>
  <style>
    body {
```

```

        background-color: lightgray;
    }
    p {
        color: green;
        font-size: 16px;
    }
    h1 {
        color: blue;
        text-align: center;
    }
</style>
</head>
<body>
    <h1>Welcome to the Page</h1>
    <p>This is a paragraph with green text and 16px font size.</p>
</body>
</html>

```

Advantages

1. **Centralized Styling:** All styles are defined in one place within the `<style>` tag, making it easier to manage for single-page websites.
2. **Improved Maintainability:** Styles are grouped together in the `<style>` tag, making updates simpler compared to Inline CSS.
3. **No External Files Needed:** Ideal for smaller projects, as it doesn't require external CSS files.
4. **Overrides External Styles:** Internal CSS can override external styles due to its higher specificity.

Disadvantages

1. **Limited to One Page:** Styles apply only to the document in which they are defined, making it unsuitable for multi-page websites.
2. **Increased Page Load Time:** The browser must load the entire HTML document, including embedded CSS, which may slow down page rendering.
3. **Not Ideal for Scalability:** Maintaining styles across multiple pages becomes inefficient.
4. **Redundancy:** Similar styles need to be duplicated on each page, leading to redundancy.

When to Use

1. **Single-Page Websites:** Best for small or one-page projects.
2. **Prototyping:** Useful for quick testing or design development.
3. **Document-Specific Styles:** When styles are specific to a single page and not needed globally.

3.External CSS

- External CSS is a technique for applying styles to HTML documents by linking an external CSS file that contains all the style rules.
- This approach allows styles to be centralized in a separate file with a .css extension, and the styles can then be applied to multiple HTML documents by linking to this external file.
- External CSS promotes consistency and maintainability, particularly in larger web projects.
- To use External CSS, the CSS file must be linked to the HTML document via the <link> tag, which is placed in the <head> section of the HTML document.

<head>

```
<link rel="stylesheet" type="text/css" href="styles.css">
```

</head>

External CSS File (styles.css):

```
body {  
    background-color: lightgray;  
}
```

```
p {  
    color: green;  
    font-size: 16px;  
}
```

Advantages

1. **Reusability:** One CSS file can be used across multiple HTML pages, ensuring consistent design.
2. **Separation of Concerns:** Keeps HTML (content) and CSS (presentation) separate, improving organization.
3. **Improved Maintainability:** Changes in one CSS file update all linked pages, simplifying maintenance.
4. **Performance Benefits:** External CSS files are cached by browsers, improving load times for returning users.

Disadvantages

1. **Dependency on External File:** Missing or broken links to the CSS file can result in an unstyled webpage.
2. **Additional HTTP Requests:** External CSS requires an extra HTTP request, which may delay initial page load.

When to Use

1. **Large Websites:** Essential for multi-page sites with consistent styling.
2. **Scalable Projects:** Best for projects that require easy updates across many pages.
3. **Professional Design:** Ideal for clean, maintainable, and scalable web development.

Comparison: Inline CSS vs Internal CSS vs External CSS

Feature	Inline CSS	Internal CSS	External CSS
Location	Defined within an individual HTML element using the style attribute	Placed within the <style> tag in the <head> section of the HTML document	Defined in a separate .css file, linked to the HTML document using the <link> tag
Scope	Affects only the specific HTML element to which it is applied	Affects the entire HTML document in which it is embedded	Can be applied to multiple HTML documents that reference the external file
Reusability	Not reusable across elements or pages; must be repeated for each element	Not reusable across pages, confined to a single document	Highly reusable across multiple pages, ensuring consistent styling across the entire site
Maintainability	Difficult to maintain for large projects as styles are scattered within HTML elements	Easier than inline, but can become cumbersome for larger websites with many pages	Highly maintainable, as changes made in a single external CSS file propagate across all linked pages
Performance	Does not impact load time; styles are applied directly to the element	Slight impact due to the inclusion of styles within the document itself	Potential delay in the initial page load, but improved performance on subsequent visits due to browser caching
Best For	Small or isolated changes, quick styling for individual elements	Small websites, single-page applications, or quick projects that don't require external files	Large, multi-page websites where consistency and scalability are important
Code Duplication	High, as styles must be defined for each element individually	Medium, as styles are defined once but can be applied to the whole page	Low, as a single external CSS file can be used across multiple pages
File Dependency	No dependency on external files, as styles are directly applied within the HTML	Relies on the internal <style> tag, which is embedded in the HTML document	Relies on the correct linking of the external CSS file, creating a dependency on an external resource
Flexibility	Very specific to each element, offering limited flexibility	More centralized but still limited to the document's scope	Most flexible, allowing centralized control over styles for an entire site

Load Time Impact	None, as the styles are applied directly to elements	Slight delay in page load, as styles are embedded within the HTML	May introduce a delay in the initial page load, but subsequent visits benefit from caching, improving performance
-------------------------	--	---	---

CSS Selector

- A **CSS Selector** is a pattern or rule used in CSS (Cascading Style Sheets) to select and target specific HTML elements on a webpage for styling.
- By using selectors, developers can apply different styles (like color, font size, margin, padding, etc.) to individual elements or groups of elements based on various attributes such as their tag names, class, ID, or even their state in the document structure.
- CSS selectors play a crucial role in controlling the appearance of a webpage. They allow developers to select elements in a way that is both efficient and precise, ensuring that styles are applied only to the desired elements without affecting others.

1. Simple Selectors

Simple selectors are the most basic form of CSS selectors. They allow you to select HTML elements based on their type, class, ID, or universal properties.

Selector Type	Definition	Syntax	Example	Explanation
Type Selector	Selects elements based on their HTML tag (type).	element { property: value; }	p { color: red; }	Applies a style to all <p> elements, changing their text color to red.
Class Selector	Selects elements with a specific class attribute. A class can be shared by multiple elements.	.class-name { property: value; }	.highlight { background-color: yellow; }	Applies a yellow background to all elements with the class highlight.
ID Selector	Selects an element with a specific id attribute. IDs should be unique within a page.	#id-name { property: value; }	#header { font-size: 24px; }	Targets the element with the ID header, setting its font size to 24px.
Universal Selector	Selects all elements on the page. Typically used to apply global styles (e.g., reset margins).	* { property: value; }	* { margin: 0; padding: 0; }	Removes default margin and padding for all elements on the page.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Selectors Example</title>
  <style>
    /* Type Selector */
    p {
      color: blue;
    }

    /* Class Selector */
    .highlight {
      background-color: yellow;
    }

    /* ID Selector */
    #header {
      font-size: 24px;
      text-align: center;
    }

    /* Universal Selector */
    * {
      margin: 0;
      padding: 0;
    }
  </style>
</head>
<body>
  <div id="header">Welcome to My Website</div>
  <p class="highlight">This paragraph has a highlighted background.</p>
```


<p>This is a normal paragraph.</p>

</body>

</html>

2. Combinator Selectors

Combinator selectors describe the relationship between different elements in an HTML structure. These selectors enable you to target elements based on their hierarchical or sibling relationship.

Selector Type	Definition	Syntax	Example	Explanation
Descendant Selector	Selects all elements that are descendants (children, grandchildren, etc.) of a specified parent element.	parent descendant { property: value; }	.container p { color: green; }	Applies green text color to all <p> elements that are descendants of an element with class container.
Child Selector	Selects only direct child elements of a specified parent.	parent > child { property: value; }	div > p { font-weight: bold; }	Applies bold text to <p> elements that are direct children of a <div> element.
General Sibling Selector	Selects all elements that are siblings (following elements) of a specified element, not necessarily adjacent.	element ~ element { property: value; }	h1 ~ p { font-style: italic; }	Applies italic font style to all <p> elements that are siblings of an <h1> element, regardless of whether they are adjacent.
Adjacent Sibling Selector	Selects the first element that directly follows a specified element.	element + element { property: value; }	h2 + p { color: blue; }	Changes the color of the first <p> element that directly follows an <h2> element to blue.

Example:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>CSS Combinator Selectors</title>
```

```
<style>
```

```
  /* DESCENDANT COMBINATOR (space) */
```

```
  .descendant p{  
    color: blue;  
  }
```

```

/* CHILD COMBINATOR (>) */
.child > p{
    color: red;
}

/* ADJACENT SIBLING COMBINATOR (+) */
.adjacent h2 + p{
    background-color: yellow;
}

/* GENERAL SIBLING COMBINATOR (~) */
.general h2 ~ p{
    border: 2px solid green;
    padding: 5px;
}

body{
    font-family: Arial, sans-serif;
    margin: 20px;
}

.box{
    border: 2px solid black;
    padding: 15px;
    margin-bottom: 20px;
}

h1{
    text-align: center;
}

</style>
</head>
<body>

<h1>CSS Combinator Selectors Example</h1>

<!-- DESCENDANT COMBINATOR -->
<div class="box descendant">
    <h2>1. Descendant Combinator (space)</h2>

    <p>Direct paragraph inside div.</p>

    <div>
        <p>Nested paragraph inside another div.</p>
    </div>
    <span>
        <p>Paragraph inside span.</p>
    </span>

    <h3>

```

All paragraphs become BLUE because they are descendants.

</h3>

</div>

<!-- CHILD COMBINATOR -->

<div class="box child">

<h2>2. Child Combinator (>)</h2>

<p>Direct child paragraph.</p>

<div>

<p>Nested paragraph inside div.</p>

</div>

<section>

<p>Paragraph inside section.</p>

</section>

<h3>

Only direct child paragraph becomes RED.

</h3>

</div>

<!-- ADJACENT SIBLING COMBINATOR -->

<div class="box adjacent">

<h2>3. Adjacent Sibling Combinator (+)</h2>

<p>This paragraph is immediately after h2.</p>

<p>This paragraph is NOT immediately after h2.</p>

<div>

<p>Paragraph inside div.</p>

</div>

<h3>

Only first paragraph gets YELLOW background.

</h3>

</div>

<!-- GENERAL SIBLING COMBINATOR -->

<div class="box general">

<h2>4. General Sibling Combinator (~)</h2>

<p>First sibling paragraph.</p>

<p>Second sibling paragraph.</p>

<div>

<p>Paragraph inside div.</p>

</div>

<p>Third sibling paragraph.</p>

<h3>

All sibling paragraphs after h2 get GREEN border.

</h3>

</div>

</body>

</html>

3. Pseudo-Classes

Pseudo-classes are used to select elements based on their state or position in the document. These selectors allow you to apply styles under certain conditions, such as when an element is hovered over or is the first child of its parent.

Pseudo-Class Type	Definition	Syntax	Example	Explanation
:hover	Selects an element when the user hovers over it.	element:hover { property: value; }	a:hover { color: red; }	Changes the color of links to red when the user hovers over them.
:nth-child()	Selects elements based on their position in a parent element.	element:nth-child(n) { property: value; }	li:nth-child(odd) { background-color: lightgray; }	Applies light gray background color to odd-numbered elements within a parent.
:first-child	Selects the first child element of a parent element.	parent:first-child { property: value; }	div:first-child { font-size: 20px; }	Sets the font size to 20px for the first child of any <div> element.
:last-child	Selects the last child element of a parent element.	parent:last-child { property: value; }	ul:last-child { color: green; }	Changes the color to green for the last child of a element.
:not()	Selects elements that do not match a specific selector.	element:not(selector) { property: value; }	div:not(.highlight) { background-color: gray; }	Applies a gray background to all <div> elements except those with the highlight class.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pseudo-Classes </title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }

    h1 {
      text-align: center;
      margin-bottom: 30px;
    }

    /* Hover */
    button:hover {
      background-color: #3498db;
      color: white;
      cursor: pointer;
    }

    /* Focus */
    input:focus {
      border: 3px solid green;
      background-color: #f0fff0;
      outline: none;
    }

    /* Visited and Link */
    a:link {
      color: #2c3e50;
      text-decoration: none;
    }

    a:visited {
      color: purple;
    }

    a:hover {
      color: red;
      text-decoration: underline;
    }

    /* First-child and Last-child */
    ul li:first-child {
      color: red;
      font-weight: bold;
```

```

}

ul li:last-child {
  color: blue;
  font-style: italic;
}

/* Nth-child */
ul li:nth-child(2) {
  background-color: yellow;
}

/* Checked */
input[type="checkbox"]:checked {
  accent-color: green;
}

/* Disabled */
input:disabled {
  background-color: #eee;
  color: #888;
}

/* Simple styling */
section {
  border: 1px solid #ccc;
  padding: 15px;
  margin-bottom: 20px;
}

label {
  margin-right: 10px;
}
</style>
</head>
<body>

<h1>CSS Pseudo-Classes selector</h1>

<!-- Hover Example -->
<section>
  <h2>Hover</h2>
  <button>Hover Over Me</button>
</section>

<!-- Focus Example -->
<section>
  <h2>Focus</h2>
  <input type="text" placeholder="Click here to focus">
</section>

<!-- Link and Visited Example -->

```

```

<section>
  <h2>Link / Visited / Hover</h2>
  <a href="https://www.google.com" target="_blank">Visit Google</a>
</section>

<!-- First-child, Last-child, Nth-child -->
<section>
  <h2>First, Last, and Nth Child</h2>
  <ul>
    <li>Apple (first-child)</li>
    <li>Mango (nth-child(2))</li>
    <li>Orange (last-child)</li>
  </ul>
</section>

<!-- Checked Example -->
<section>
  <h2>Checked</h2>
  <label><input type="checkbox"> Option 1</label>
  <label><input type="checkbox"> Option 2</label>
</section>

<!-- Disabled Example -->
<section>
  <h2>Disabled</h2>
  <input type="text" placeholder="Disabled Input" disabled>
</section>

</body>
</html>

```

4. Pseudo-Elements

Pseudo-elements allow you to style specific parts of an element, such as inserting content before or after an element's content.

Pseudo-Element Type	Definition	Syntax	Example	Explanation
::before	Inserts content before the content of an element.	element::before { content: "text"; property: value; }	p::before { content: "Note: "; font-weight: bold; }	Adds the text "Note: " before every <p> element and makes it bold.
::after	Inserts content after the content of an element.	element::after { content: "text"; property: value; }	p::after { content: "(End of paragraph)"; color: gray; }	Adds the text "(End of paragraph)" after every <p> element and sets its color to gray.

::first-letter	Selects and applies styles to the first letter of an element.	element::first-letter { property: value; }	p::first-letter { font-size: 2em; font-weight: bold; }	Applies a larger and bold font style to the first letter of every <p> element.
::first-line	Selects and applies styles to the first line of an element.	element::first-line { property: value; }	p::first-line { color: blue; }	Changes the color of the first line of text in every <p> element to blue.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pseudo-Elements Example</title>
  <style>
    /* ::before */
    p::before {
      content: "Note: ";
      font-weight: bold;
    }

    /* ::after */
    p::after {
      content: " (End of paragraph)";
      color: gray;
    }

    /* ::first-letter */
    p::first-letter {
      font-size: 2em;
      font-weight: bold;
    }
```



```

/* ::first-line */
p::first-line {
  color: blue;
}
</style>
</head>
<body>
  <p>This is a paragraph where the first letter is styled, and the first line is blue.</p>
</body>
</html>

```

5. Attribute Selectors

Attribute selectors target elements based on the presence or value of their attributes. They provide a powerful way to select elements with specific attribute values.

Attribute Selector Type	Definition	Syntax	Example	Explanation
[attribute="value"]	Selects elements with a specific attribute value.	element[attribute="value"] { property: value; }	a[href="https://example.com"] { color: blue; }	Applies a blue color to links with href="https://example.com".
[attribute^="value"]	Selects elements whose attribute value starts with a specified string.	element[attribute^="value"] { property: value; }	img[src^="https://"] { border: 2px solid green; }	Targets elements with src starting with "https://", applying a green border.

[attribute\$="value"]	Selects elements whose attribute value ends with a specified string.	element[attribute\$="value"] { property: value; }	img[src\$=".png"] { border: 2px solid red; }	Targets elements with src ending with .png, applying a red border.
[attribute*="value"]	Selects elements whose attribute value contains a specified string.	element[attribute*="value"] { property: value; }	a[href*="example"] { color: orange; }	Applies an orange color to all links whose href contains the string "example".

Example:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Attribute Selector Example</title>
<style>
  /* Selects the input element with type="text" */
  input[type="text"] {
    border: 2px solid red;
    background-color: #f9f9f9;
  }

  /* Selects the input elements with a placeholder attribute containing "name" */
  input[placeholder*="name"] {

```

```
border-color: green;
```

```
}
```

```
/* Selects all links whose href attribute ends with .pdf */
```

```
a[href$=".pdf"] {
```

```
color: blue;
```

```
text-decoration: underline;
```

```
}
```

```
/* Selects all images whose src attribute starts with "https://" */
```

```
img[src^="https://"] {
```

```
border: 3px solid green;
```

```
}
```

```
/* Selects all links whose href attribute contains the word "example" */
```

```
a[href*="example"] {
```

```
color: orange;
```

```
font-weight: bold;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<!-- Input fields with different attributes -->
```

```
<input type="text" placeholder="Enter your name">
```

```
<input type="password" placeholder="Enter your password">
```

```
<input type="text" placeholder="Enter your email address">
```

```
<!-- Links with various href values -->
```

```
<a href="https://www.example.com/document.pdf">Download PDF 1</a>
```

```
<a href="https://www.test.com/page.html">Go to Test Page</a>
```

```
<a href="https://www.example.com/info.pdf">Download PDF 2</a>
```

```
<!-- Images with different src attributes -->
```

```

```

```

```

```

```

```
</body>
```

```
</html>
```

CSS Colors

- CSS colors allow designers and developers to style web pages by specifying colors for various properties such as text, backgrounds, borders, etc.
- Colors play a significant role in the user experience, design aesthetics, and branding of a website. Using the right color values and combinations is essential for visual clarity and accessibility.

Types of CSS Color Specifications (CSS Color Models)

CSS supports several methods to define colors. These methods offer varying levels of flexibility and precision.

- 1. Named Colors:** Predefined color names recognized by all web browsers.

Syntax: color: color-name;

Examples: color: red; background-color: lightblue;

Pros: Easy to use, human-readable.

Cons: Limited selection compared to other methods.

black	bisque	forestgreen	slategrey
dimgray	darkorange	limegreen	lightsteelblue
dimgrey	burlywood	darkgreen	cornflowerblue
gray	antiquewhite	green	royalblue
grey	tan	lime	ghostwhite
darkgray	navajowhite	seagreen	lavender
darkgrey	blanchedalmond	mediumseagreen	midnightblue
silver	papayawhip	springgreen	navy
lightgray	moccasin	mintcream	darkblue
lightgrey	orange	mediumspringgreen	mediumblue
gainsboro	wheat	mediumaquamarine	blue
whitesmoke	oldlace	aquamarine	slateblue
white	floralwhite	turquoise	darkslateblue
snow	darkgoldenrod	lightseagreen	mediumslateblue
rosybrown	goldenrod	mediumturquoise	mediumpurple
lightcoral	cornsilk	azure	rebeccapurple
indianred	gold	lightcyan	blueviolet
brown	lemonchiffon	paleturquoise	indigo
firebrick	khaki	darkslategray	darkorchid
maroon	palegoldenrod	darkslategrey	darkviolet
darkred	darkkhaki	teal	mediumorchid
red	ivory	darkcyan	thistle
mistyrose	beige	aqua	plum
salmon	lightyellow	cyan	violet
tomato	lightgoldenrodyellow	darkturquoise	purple
darksalmon	olive	cadetblue	darkmagenta
coral	yellow	powderblue	fuchsia
orangered	olivedrab	lightblue	magenta
lightsalmon	yellowgreen	deepskyblue	orchid
sienna	darkolivegreen	skyblue	mediumvioletred
seashell	greenyellow	lightskyblue	deeppink
chocolate	chartreuse	steelblue	hotpink
saddlebrown	lawngreen	aliceblue	lavenderblush
sandybrown	honeydew	dodgerblue	palevioletred
peachpuff	darkseagreen	lightslategray	crimson
peru	palegreen	lightslategrey	pink
linen	lightgreen	slategray	lightpink

2 .RGB (Red, Green, Blue) : Colors specified by their red, green, and blue components. Each component can range from 0 to 255.

Syntax: color: rgb(red, green, blue);

Examples:

color: rgb(255, 0, 0); (Red)

background-color: rgb(0, 255, 0); (Green)

Pros: Direct control over the color intensity.

Cons: Does not include transparency by default.

3.RGBA (Red, Green, Blue, Alpha): Similar to RGB, but with an added alpha channel for transparency. The alpha value is a number between 0 (fully transparent) and 1 (fully opaque).

Syntax: color: rgba(red, green, blue, alpha);

Examples:

color: rgba(255, 0, 0, 0.5); (Semi-transparent Red)

background-color: rgba(0, 255, 0, 0.3); (Semi-transparent Green)

Pros: Provides transparency control.

Cons: Requires understanding of alpha blending.

4.HEX (Hexadecimal): Colors specified as a six-digit hexadecimal value, representing the red, green, and blue components.

Syntax: color: #RRGGBB;

Examples:

color: #FF0000; (Red)

background-color: #00FF00; (Green)

Pros: Compact and widely used in web design.

Cons: No direct control over transparency.

5. HSL (Hue, Saturation, Lightness): HSL model specifies colors based on their hue , saturation ,and lightness.

Hue: The type of color (represented as an angle on the color wheel, ranging from 0 to 360 degrees).

Saturation: The intensity or vividness of the color, expressed as a percentage (0% = gray, 100% = full color).

Lightness: The lightness or darkness of the color, also expressed as a percentage (0% = black, 100% = white, 50% = normal).

Syntax: color: hsl(hue, saturation%, lightness%);

Examples:

color: hsl(0, 100%, 50%); (Red)

background-color: hsl(120, 100%, 50%); (Green)

Pros: Intuitive and useful for color manipulation.

Cons: Less precise for certain color manipulations.

6. HSLA (Hue, Saturation, Lightness, Alpha)

- **HSLA** stands for **Hue, Saturation, Lightness, and Alpha**. It is an extension of the **HSL** model (Hue, Saturation, and Lightness) with the addition of an **Alpha** channel, which controls the transparency of the color. The **Alpha** value defines the opacity, allowing the color to be semi-transparent.
- **HSLA** is useful for web design as it offers a more intuitive way to handle color opacity (transparency), especially when creating background effects or hover states, making it a versatile tool for modern CSS.

Syntax : color: hsla(hue, saturation%, lightness%, alpha);

hue: The color type, represented as an angle on the color wheel (0° to 360°). This determines the base color.

saturation: The intensity of the color, expressed as a percentage (0% = gray, 100% = full color).

lightness: The lightness or darkness of the color, also expressed as a percentage (0% = black, 100% = white, 50% = normal).

alpha: The opacity of the color, expressed as a value between 0 (fully transparent) and 1 (fully opaque).

Pros: Provides transparency and intuitive control.

Cons: Similar to RGBA, requires understanding of transparency effects.

CSS Color Properties

- CSS provides a wide range of color-related properties that allow developers to apply specific color effects to various elements within a webpage.
- These properties control the color of text, backgrounds, borders, outlines, shadows, and other visual aspects of the page, contributing to its overall design and user experience.

Property	Description	Syntax Example	Common Use Cases
color	Specifies the color of the text within an element.	color: red;	Text color for paragraphs, headings, links, etc.
background-color	Sets the background color of an element.	background-color: #3498db;	Background color for divs, sections, body, etc.
border-color	Defines the color of an element's border.	border-color: green;	Border color for buttons, inputs, images, etc.

outline-color	Specifies the color of the outline (drawn outside the element's border).	outline-color: blue;	Outline color for focused or active elements.
text-decoration-color	Sets the color of text decoration (e.g., underline, line-through).	text-decoration-color: yellow;	Underline or strike-through decoration in links or text.
box-shadow	Defines the color of the shadow applied to an element.	box-shadow: 4px 4px 10px rgba(0, 0, 0, 0.5);	Shadow effect for buttons, divs, images, etc.
background	A shorthand property for setting both background color and background image.	background: #3498db url('image.jpg') no-repeat;	Combined background color and image for divs, sections.
opacity	Sets the transparency of an element. Values range from 0 (fully transparent) to 1 (fully opaque).	opacity: 0.5;	To create semi-transparent backgrounds or elements.
filter	Applies graphical effects like blur or brightness, which can affect color.	filter: brightness(0.5);	Adjusts visual appearance (e.g., grayscale, blur) of an element.
accent-color	Sets a default color for UI controls such as sliders, checkboxes, and radio buttons.	accent-color: #3498db;	Defines the accent color for interactive form elements.
border-bottom-color	Specifies the color of the bottom border of an element.	border-bottom-color: #ff6347;	For styling the bottom border of elements like divs or links.
border-left-color	Specifies the color of the left border of an element.	border-left-color: #ff6347;	For styling the left border of elements like divs or sections.
border-right-color	Specifies the color of the right border of an element.	border-right-color: #ff6347;	For styling the right border of elements like divs or sections.
border-top-color	Specifies the color of the top border of an element.	border-top-color: #ff6347;	For styling the top border of elements like divs or sections.

Example:

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CSS Color Properties Example</title>
<style>
  /* Text Color */
  h1 {
    color: #3498db; /* Blue text */
  }

  /* Background Color */
  body {
    background-color: #f0f0f0; /* Light gray background */
    font-family: Arial, sans-serif;
  }

```



```

}

/* Border Color */
button {
  border: 2px solid #2ecc71; /* Green border */
  padding: 10px 20px;
  background-color: white;
  cursor: pointer;
}

/* Text Decoration Color */
a {
  text-decoration: underline;
  text-decoration-color: yellow; /* Yellow underline */
  color: #8e44ad; /* Purple link color */
}

/* Box Shadow */
.card {
  box-shadow: 5px 5px 15px rgba(0, 0, 0, 0.3); /* Semi-transparent shadow */
  padding: 20px;
  background-color: #fff;
  margin: 20px;
  border-radius: 8px;
}

/* Opacity */
.transparent-box {
  background-color: #3498db;
  opacity: 0.5; /* 50% transparency */
  padding: 20px;
  color: white;
  border-radius: 8px;
}

/* Border Bottom Color */
.special-border {
  border-bottom: 4px solid #ff6347; /* Red bottom border */
  padding-bottom: 10px;
  font-weight: bold;
}

/* Filter */
img {
  width: 100%;
  max-width: 700px;
  filter: brightness(0.5); /* Makes image darker */
  border: 5px solid #ec059f;
}

/* Accent Color */
input[type="checkbox"] {

```

```
    accent-color: #bff304; /* green checkbox */
  }
</style>
</head>
<body>
```

```
<h1>CSS Color Properties Example</h1>
```

```
<div class="card">
  <p>This is a card with a shadow effect.</p>
</div>
```

```
<p class="special-border">This paragraph has a red bottom border.</p>
```

```
<div class="transparent-box">
  <p>This box has 50% opacity.</p>
</div>
```

```
<button>Click Me!</button>
```

```
<p>
  <a href="#">Hover over this link to see the yellow underline.</a>
</p>
```

```

```

```
<label>
  <input type="checkbox"> Check me!
</label>
</body>
</html>
```

CSS Color Properties Example

This is a card with a shadow effect.

This paragraph has a red bottom border.

This box has 50% opacity.

Click Me!

Hover over this link to see the yellow underline.



✓ Check me!

CSS Background

- The background property in CSS is used to control the presentation of an element's background, such as color, image, position, size, and behavior.
- It is an essential tool in web design for setting the visual foundation of a webpage, enhancing the user experience with aesthetically pleasing and functional background elements.
- The background property is shorthand for setting multiple background-related properties at once, while each individual background property has specific functionalities.
- It includes the following primary sub-properties:

1. **background-color**
2. **background-image**
3. **background-position**
4. **background-size**
5. **background-repeat**
6. **background-attachment**
7. **background-origin**
8. **background-clip**

- Each property allows you to set distinct values that collectively define how the background of an element should behave.

1. background-color

The background-color property specifies the background color of an element.

Possible Value	Explanation	Example
color names	Predefined color names like red, blue, green, etc.	background-color: red;
hex values	A 6-digit hexadecimal color code representing RGB color.	background-color: #3498db;
rgb()	Defines colors using the RGB (Red, Green, Blue) color model. The values range from 0 to 255.	background-color: rgb(52, 152, 219);
rgba()	RGB with Alpha channel for transparency. Values range from 0 to 255 for RGB, and 0 to 1 for alpha.	background-color: rgba(52, 152, 219, 0.5);
hsl()	Defines colors using Hue, Saturation, and Lightness values. Hue is between 0 and 360, saturation and lightness are percentages.	background-color: hsl(210, 100%, 50%);
hsla()	HSL with Alpha channel for transparency. a is a value between 0 (fully transparent) and 1 (fully opaque).	background-color: hsla(210, 100%, 50%, 0.5);

2. background-image: The background-image property specifies an image as the background for an element.

Possible Value	Explanation	Example
url()	Specifies the URL of the image to be used as the background.	background-image: url('image.jpg');
none	Specifies that no background image should be applied to the element.	background-image: none;

3. background-position: The background-position property determines the position of the background image within the element.

Possible Value	Explanation	Example
left, center, right	Specifies horizontal alignment. Can be used alone or in combination with vertical values.	background-position: center;
top, center, bottom	Specifies vertical alignment. Can be used alone or in combination with horizontal values.	background-position: top right;
percentage	Specifies the position relative to the element's dimensions (e.g., 50% 50% means center).	background-position: 50% 50%;
length (px, em, etc.)	Specifies the position in terms of pixels or other length units (e.g., px, em).	background-position: 20px 30px;

4. background-size: The background-size property controls the size of the background image.

Possible Value	Explanation	Example
auto	The background image retains its original size.	background-size: auto;
cover	Scales the background image to fully cover the element, potentially cutting off some of the image.	background-size: cover;
contain	Scales the background image to ensure it fits entirely within the element. It may leave empty space.	background-size: contain;
length values	Specifies explicit width and height for the background image. Can be in px, em, %, etc.	background-size: 100px 50px;

5. background-repeat: The background-repeat property specifies whether and how the background image is repeated.

Possible Value	Explanation	Example
repeat	The background image is repeated both horizontally and vertically.	background-repeat: repeat;
no-repeat	The background image is not repeated (only one image is shown).	background-repeat: no-repeat;
repeat-x	The background image is repeated only horizontally.	background-repeat: repeat-x;
repeat-y	The background image is repeated only vertically.	background-repeat: repeat-y;

6. background-attachment: The background-attachment property determines how the background image scrolls with the page.

Possible Value	Explanation	Example
scroll	The background image scrolls with the page content.	background-attachment: scroll;
fixed	The background image stays fixed in place even when the page is scrolled (parallax effect).	background-attachment: fixed;
local	The background image scrolls with the element's content.	background-attachment: local;

7. background-origin : The background-origin property specifies where the background image is positioned relative to.

Possible Value	Explanation	Example
content-box	The background image is positioned relative to the content box (inner most area).	background-origin: content-box;
padding-box	The background image is positioned relative to the padding area.	background-origin: padding-box;
border-box	The background image is positioned relative to the border area.	background-origin: border-box;

8. background-clip: The background-clip property defines the area within the element where the background is painted.

Possible Value	Explanation	Example
border-box	The background is painted within the border area (the background extends to the border).	background-clip: border-box;
padding-box	The background is clipped to the padding area.	background-clip: padding-box;
content-box	The background is clipped to the content area (does not extend into padding or border).	background-clip: content-box;

CSS Borders

- Borders in CSS are used to define the perimeter of an element. They help in visually separating content, enhancing the structure of layouts, and improving user experience.
- CSS provides a variety of ways to control the appearance, style, width, and color of borders.
- This makes it a powerful tool for web designers to manipulate the boundaries of HTML elements.

CSS Border Properties

1. border (Shorthand Property): The border property is a shorthand way to set the width, style, and color of the borders on all sides of an element. This is the most common property used to create borders.

Syntax:

```
element {  
  border: [border-width] [border-style] [border-color];  
}
```

- **border-width:** Specifies the thickness of the border.
- **border-style:** Defines the type of border (e.g., solid, dashed).
- **border-color:** Sets the color of the border.

Example:

```
div {  
  border: 5px solid #3498db;  
}
```

This applies a **5px solid blue border** around the <div> element.

2. border-width: The border-width property is used to define the thickness of the border around an element. It accepts length values such as px, em, rem, %, or predefined keywords like thin, medium, and thick.

Syntax:

```
element {  
  
  border-width: [value];  
}
```

```
}
```

Possible Values:

- **thin:** A thin border.
- **medium:** A medium-width border (default).
- **thick:** A thick border.
- **Length units:** px, em, rem, etc.

Example:

```
div {  
  border-width: 3px;  
}
```

This applies a **3px** wide border around the <div> element.

3. border-style: The border-style property defines the style of the border. It determines whether the border is solid, dotted, dashed, or any other predefined type.

Syntax:

```
element {  
  border-style: [style];  
}
```

Possible Values:

1. **none:** No border is applied.
2. **solid:** A solid line.
3. **dashed:** A series of short lines.
4. **dotted:** A series of dots.
5. **double:** Two solid lines with space between them.
6. **groove:** A 3D grooved effect.
7. **ridge:** A raised 3D effect.

Example:

```
div {  
  border-style: dashed;  
}
```

This applies a **dashed border** around the <div> element.

4. border-color: The border-color property allows you to set the color of the border. It can accept color values in several formats, including **hex**, **RGB**, **RGBA**, **HSL**, **HSLA**, and **named colors**.

Syntax:

```
element {  
  border-color: [color];  
}
```

Example:

```
div {  
  border-color: #3498db; /* Blue border */  
}
```

You can specify individual colors for each side of the border:

```
div {  
  border-color: red green blue yellow; /* top, right, bottom, left */  
}
```

5. border-radius: The border-radius property is used to round the corners of an element's border. This can be used to create circles or rounded rectangles by controlling the radius of the element's corners.

Syntax:

```
element {  
  border-radius: [value];  
}
```

- **Length values** (e.g., px, em, %).

Example:

```
div {  
  border-radius: 15px;  
}
```

This applies **rounded corners** with a **15px** radius to the <div> element.

You can also specify different radii for each corner:

```
div {  
  border-radius: 10px 20px 30px 40px; /* top-left, top-right, bottom-right, bottom-left */  
}
```

6. border-top, border-right, border-bottom, border-left: These properties allow you to set the border for individual sides of an element (top, right, bottom, or left). This gives more control over the design of borders on each side.

Syntax:

```
element {  
  border-top: [width] [style] [color];  
  border-right: [width] [style] [color];  
  border-bottom: [width] [style] [color];  
  border-left: [width] [style] [color];  
}
```

Example:

```
div {  
  border-top: 5px solid red; /* Top border */  
  border-right: 5px dashed green; /* Right border */  
  border-bottom: 5px dotted blue; /* Bottom border */  
  border-left: 5px double black; /* Left border */  
}
```

7. border-collapse and border-spacing: These properties are used for tables to control the behavior of borders between table cells.

1. **border-collapse:** Determines whether table borders should collapse into a single border or remain separated.
 - **collapse:** Borders collapse into a single border.
 - **separate:** Borders remain separated.
2. **border-spacing:** Specifies the space between borders of adjacent cells when border-collapse is set to separate.

Example:

```
table {  
  border-collapse: collapse;  
  border-spacing: 5px;  
}
```

Example: CSS Borders

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<title>CSS Borders Example</title>  
<style>  
  /* Box with Solid Border */
```

```
.box-solid {
  border: 5px solid #3498db;
  padding: 20px;
  margin: 20px;
}

/* Box with Dashed Border */
.box-dashed {
  border: 3px dashed #ff6347;
  padding: 20px;
  margin: 20px;
}

/* Box with Rounded Corners */
.box-rounded {
  border: 5px solid #2ecc71;
  border-radius: 15px;
  padding: 20px;
  margin: 20px;
}

/* Box with Different Border on Each Side */
.box-custom {
  border-top: 5px solid #e74c3c;
  border-right: 5px dashed #f39c12;
  border-bottom: 5px dotted #8e44ad;
  border-left: 5px double #1abc9c;
  padding: 20px;
  margin: 20px;
}
</style>
</head>
<body>

<div class="box-solid">
  <p>This box has a solid blue border.</p>
</div>

<div class="box-dashed">
  <p>This box has a dashed red border.</p>
</div>

<div class="box-rounded">
  <p>This box has rounded corners with a green border.</p>
</div>

<div class="box-custom">
  <p>This box has different border styles on each side.</p>
</div>

</body>
</html>
```



CSS Margin

- The margin property in CSS is used to create space around an element, outside of its border. It controls the amount of space between an element and its neighboring elements on a webpage.
- Margins are transparent, which means they don't have a background color.
- Margins help in separating elements, making a webpage look clean and organized. They are essential for managing the layout of a page and preventing elements from touching each other.
- margins can set in several ways:

Syntax	Description	Example
margin: [value];	Single value applies to all four sides.	margin: 20px;
margin: [top-bottom] [left-right];	Two values: first for top and bottom, second for left and right.	margin: 10px 20px;
margin: [top] [left-right] [bottom];	Three values: first for top, second for left and right, third for bottom.	margin: 10px 20px 30px;
margin: [top] [right] [bottom] [left];	Four values: first for top, second for right, third for bottom, fourth for left.	margin: 10px 20px 30px 40px;

CSS Margin Properties

Property	Description	Possible Values	Example	Use Case
margin	Sets the margin for all four sides of an element.	Length (e.g., px, em, %), auto	margin: 20px;	Apply 20px margin on all sides.
margin-top	Sets the margin on the top side of an element.	Length, auto	margin-top: 30px;	Add 30px margin at the top.
margin-right	Sets the margin on the right side of an element.	Length, auto	margin-right: 15px;	Add 15px margin on the right.
margin-bottom	Sets the margin on the bottom side of an element.	Length, auto	margin-bottom: 25px;	Add 25px margin at the bottom.
margin-left	Sets the margin on the left side of an element.	Length, auto	margin-left: 10px;	Add 10px margin on the left.

Example: <!DOCTYPE html>

```
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CSS Margin Example</title>
<style>
  /* Box with margin applied to all sides */
  .box1 {
    margin: 20px;
    background-color: lightblue;
    padding: 20px;
    font-size: large;
  }

  /* Box with top and bottom margin, left and right margin auto */
  .box2 {
    width: 50%;
    margin: 30px auto;
    background-color: lightgreen;
    padding: 20px;
    font-size: large;
  }

  /* Box with margin applied individually to top, right, bottom, and left */
  .box3 {
    margin: 10px 20px 30px 40px;
    background-color: lightcoral;
    padding: 20px;
    font-size: large;
  }

</style>
</head>
<body>

<div class="box1">
  <p>This box has a 20px margin on all sides.</p>
</div>

<div class="box2">
  <p>This box has a margin of 30px top and bottom, and auto left and right (centering it horizontally).</p>
</div>

<div class="box3">
  <p>This box has different margin values for each side (top: 10px, right: 20px, bottom: 30px, left:
40px).</p>
</div>

</body>
</html>
```

This box has a 30px margin on all sides.

This box has a margin of 30px top and bottom, and auto left and right (centering it horizontally).

This box has different margin values for each side (top: 10px, right: 30px, bottom: 30px, left: 40px).

CSS Padding

- The padding property in CSS is used to create space inside an element, between the element's content and its border. It controls the space between the content (text, images, etc.) and the border of the element.
- Unlike margins (which are outside the border), padding is applied inside the element's boundary.
- Padding ensures that content within an element does not touch the borders. It makes the content more readable and visually appealing by providing internal spacing.
- Padding increases the **size of the element**, meaning that when padding is added, the element's total width and height grow (including the content and padding area).
- Padding does not overlap with other elements or affect the layout of other elements like margins do.
- padding can set in several ways:

Syntax	Description	Example
padding: [value];	Single value applies to all four sides.	padding: 20px;
padding: [top-bottom] [left-right];	Two values: first for top and bottom, second for left and right.	padding: 10px 20px;
padding: [top] [left-right] [bottom];	Three values: first for top, second for left and right, third for bottom.	padding: 10px 20px 30px;
padding: [top] [right] [bottom] [left];	Four values: first for top, second for right, third for bottom, fourth for left.	padding: 10px 20px 30px 40px;

CSS Padding Properties

Property	Description	Possible Values	Example	Use Case
padding	Sets the padding for all four sides of an element.	Length (e.g., px, em, %), auto	padding: 20px;	Applies 20px padding on all sides.
padding-top	Sets the padding on the top side of an element.	Length, auto	padding-top: 30px;	Adds 30px padding at the top.
padding-right	Sets the padding on the right side of an element.	Length, auto	padding-right: 15px;	Adds 15px padding on the right.
padding-bottom	Sets the padding on the bottom side of an element.	Length, auto	padding-bottom: 25px;	Adds 25px padding at the bottom.
padding-left	Sets the padding on the left side of an element.	Length, auto	padding-left: 10px;	Adds 10px padding on the left.

Padding vs Margin

Aspect	Padding	Margin
Placement	Inside the element, between the content and the border.	Outside the element, between the border and adjacent elements.
Effect on Size	Increases the element's total width and height.	Does not affect the element's size but affects the layout by creating space around the element.
Visual Appearance	Adds space inside the element's border, making it less crowded.	Adds space outside the element, distancing it from other elements.

Example: Using CSS Padding

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CSS Padding Example</title>
<style>
  /* Box with padding applied to all sides */
  .box1 {
    padding: 20px;
    background-color: lightblue;
    border: 2px solid blue;
  }

  /* Box with top and bottom padding, left and right padding auto */
  .box2 {
    width: 50%;
    padding: 30px auto;
    background-color: lightgreen;
    border: 2px solid green;
  }

  /* Box with padding applied individually to top, right, bottom, and left */
  .box3 {
    padding: 10px 20px 30px 40px;
    background-color: lightcoral;
    border: 2px solid red;
  }
</style>
</head>
<body>

<div class="box1">
  <p>This box has a 20px padding on all sides.</p>
</div>

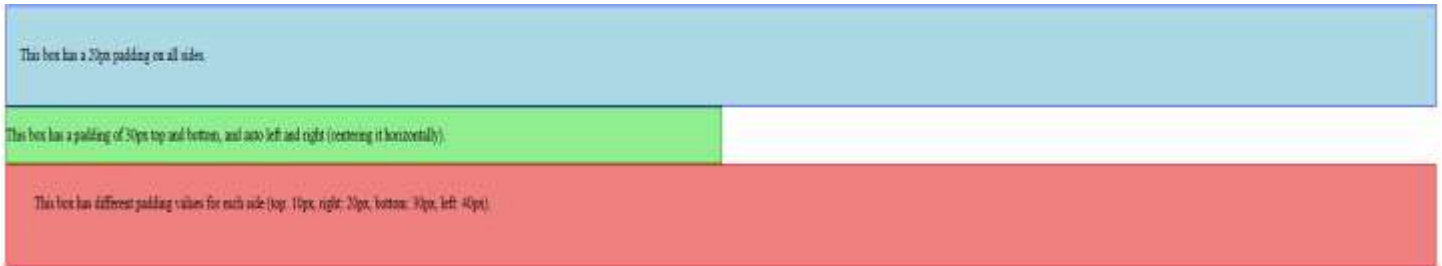
<div class="box2">
  <p>This box has a padding of 30px top and bottom, and auto left and right (centering it horizontally).</p>
</div>
```

```

<div class="box3">
  <p>This box has different padding values for each side (top: 10px, right: 20px, bottom: 30px, left:
40px).</p>
</div>

</body>
</html>

```



CSS Height and Width

- **CSS Height and Width** are essential properties in web design used to control the size and dimensions of elements on a webpage.
- These properties define how tall and how wide an element will be displayed in the browser window.
- Understanding and manipulating these properties is critical in creating responsive, structured, and visually appealing web layouts.
- **Height** controls the vertical size of an element.
- **Width** controls the horizontal size of an element.
- Together, they help define the overall size of elements like divs, images, videos, buttons, and other content containers.
- **Syntax**

```

/* Basic Syntax */
height: [value];
width: [value];

```

- value can be specified in various units such as px, %, em, rem, vh, vw, etc.

Units for Height and Width

Unit	Description	Example
px	Represents the size in pixels. It is the most common unit used in web design.	height: 200px;
%	Percentage relative to the parent element's height or width.	width: 50%;
em	Relative to the font size of the element. 1em = the current font size.	width: 5em;
rem	Relative to the font size of the root element (usually <html>).	width: 10rem;
vh	Viewport height. 1vh = 1% of the height of the viewport (browser window).	height: 50vh;
vw	Viewport width. 1vw = 1% of the width of the viewport (browser window).	width: 50vw;

Types of Height and Width Values

Fixed Height and Width: Set a precise value (in pixels, em, etc.).

Example:

```
.box {  
    width: 200px;  
    height: 150px;  
}
```

Percentage-Based Height and Width: Set relative values based on the parent element's dimensions.

Example:

```
.container {  
    width: 50%;  
    height: 60%;  
}
```

Viewport-Based Height and Width: Set the size relative to the viewport (browser window).

Example:

```
.responsive-box {  
    width: 50vw;  
    height: 30vh;  
}
```

Example:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<title>CSS Height and Width Example</title>  
<style>  
    /* Fixed dimensions */  
    .box1 {  
        width: 300px;  
        height: 200px;  
        background-color: lightblue;  
    }  
  
    /* Percentage-based */  
    .box2 {  
        width: 50%;  
        height: 60%;  
        background-color: lightgreen;  
    }  
  
    /* Viewport-based */  
    .box3 {  
        width: 50vw;  
        height: 30vh;  
        background-color: lightcoral;  
    }  
  
    /* Aspect ratio */  
    .aspect-ratio-container {  
        width: 100%;  
        height: 0;  
        padding-bottom: 56.25%;  
    }  
</style>
```

```
background-color: lightyellow;
position: relative;
}

.aspect-ratio-container img {
position: absolute;
width: 100%;
height: 100%;
}
</style>
</head>
<body>

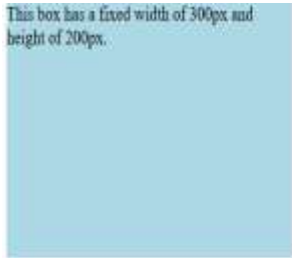
<div class="box1">
  <p>This box has a fixed width of 300px and height of 200px.</p>
</div>

<div class="box2">
  <p>This box has a width of 50% and height of 60% of its parent container.</p>
</div>

<div class="box3">
  <p>This box has a responsive width of 50vw and height of 30vh.</p>
</div>

<div class="aspect-ratio-container">
  
</div>

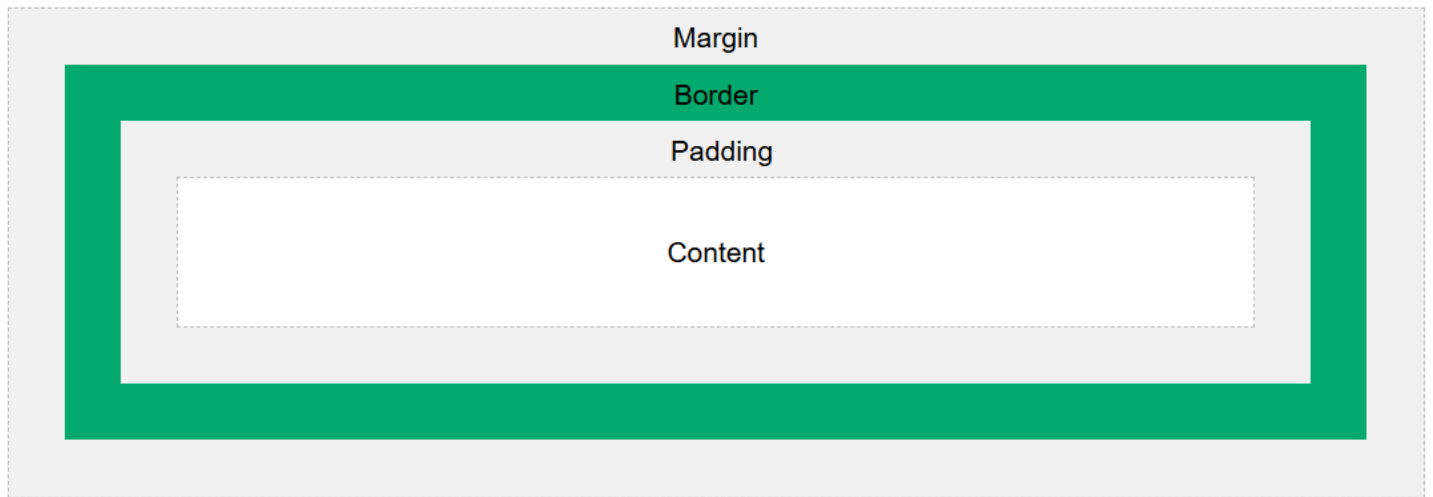
</body>
</html>
```



The CSS Box Model

- The **CSS Box Model** is a fundamental concept in web design and layout. It describes the rectangular boxes generated for every element on a webpage, which consists of several parts that determine the element's size and positioning.
- Every element in HTML is treated as a box, and the CSS Box Model defines the structure and the properties of these boxes.
- Understanding the CSS Box Model is crucial for controlling the layout of elements and avoiding unexpected behaviors in a web page design. It allows developers to define padding, borders, margins, and dimensions (width and height) in a structured way.
- The CSS Box Model is composed of **four parts**:

Part	Description	Example
Content	The actual content of the box (text, images, etc.), defined by the width and height properties.	width: 200px; height: 150px;
Padding	The space between the content and the border. Padding is inside the element and adds to the element's total size.	padding: 20px;
Border	Surrounds the padding (if any) and content. It defines the boundary of the box. The border can have width, style, and color.	border: 2px solid #000;
Margin	The outermost space around the box, separating the element from other elements. The margin does not affect the element's dimensions.	margin: 10px;



The total size of an element is calculated as follows:

Total Width = Content Width + Left Padding + Right Padding + Left Border + Right Border + Left Margin + Right Margin

Total Height = Content Height + Top Padding + Bottom Padding + Top Border + Bottom Border + Top Margin + Bottom Margin

CSS Outline

- An **outline** is a line drawn around an element outside its borders, providing a visual distinction without affecting the element's dimensions or layout.
- Unlike borders, the outline can overlap other content and does not affect the element's total width or height.
- The **outline** property consists of several sub-properties that define its appearance:

Property	Description
outline-style	Specifies the style of the outline (solid, dotted, dashed, etc.).
outline-color	Specifies the color of the outline.
outline-width	Specifies the width (thickness) of the outline.
outline-offset	Adds space between the outline and the element's border, shifting the outline outward.
Outline	A shorthand property for setting all the outline properties at once.

- The **outline-style** property defines the style of the outline.

Value	Description
dotted	Defines a dotted outline.
dashed	Defines a dashed outline.
solid	Defines a solid outline.
double	Defines a double outline.
groove	Defines a 3D grooved outline.
ridge	Defines a 3D ridged outline.
inset	Defines a 3D inset outline.
outset	Defines a 3D outset outline.
none	Defines no outline.
hidden	Defines a hidden outline.

- The **outline-color** property specifies the color of the outline. It can accept various color formats:

Value Type	Description
Color Name	e.g., red, blue, green.
Hex Code	e.g., #ff0000 (red).
RGB	e.g., rgb(255, 0, 0) (red).
RGBA	e.g., rgba(255, 0, 0, 0.5) (red, 50% opacity).

- The **outline-width** property specifies the thickness of the outline. It can take values in absolute units (such as px, em, etc.) or use keywords:

Value	Description
thin	Specifies a thin outline.
medium	Specifies a medium-width outline.
thick	Specifies a thick outline.
length	Specific length values like 2px or 0.1em.

- The **outline-offset** property creates space between the element's border and the outline. This property allows you to move the outline away from the element, providing a **visual buffer** between the element's border and the outline.

```
p {  
    outline: 2px solid red;  
    outline-offset: 5px; /* Space between outline and element */  
}
```

- The **outline** property is a shorthand for setting **outline-width**, **outline-style**, and **outline-color** in one line.

```
/* Shorthand for outline properties */  
p {  
    outline: 2px solid green; /* 2px width, solid style, green color */  
}
```

Example:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>CSS Margin, Padding, Border, and Outline Example</title>  
<style>  
  /* General styles for the element */  
  .element {  
    margin: 30px;      /* Space outside the border */  
    padding: 20px;     /* Space inside the border */  
    border: 5px solid blue; /* Blue border around the element */  
    outline: 3px dashed red; /* Dashed red outline outside the border */  
    outline-offset: 10px;  
    background-color: lightgray; /* Background color for better visibility */  
    width: 200px;      /* Set a width to the element */  
  }
```

```

        height: 150px;    /* Set a height to the element */
    }
</style>
</head>
<body>

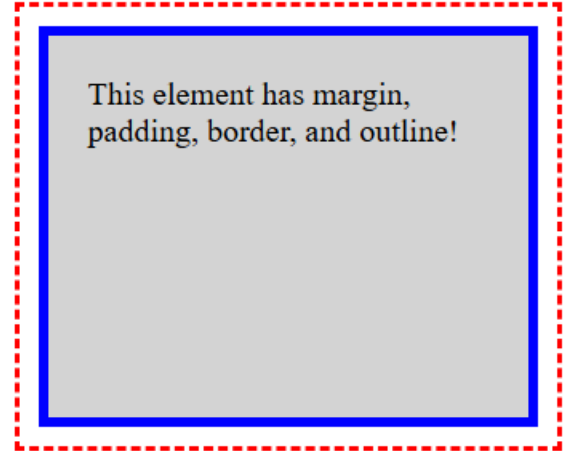
    <h1>CSS outline</h1>

    <div class="element">
        This element has margin, padding, border, and outline!
    </div>

</body>
</html>

```

CSS Outline



CSS Text Attributes

In CSS, the **text** attributes allow you to control the appearance and layout of text. These properties are crucial for styling text content, including things like **font** properties, **alignment**, **spacing**, **decoration**, and more.

1. Font Properties

Property	Value	Explanation
font-family	serif	A font with small lines at the ends of characters, like Times New Roman.
	sans-serif	A font without extra lines at the ends of characters, like Arial.
	monospace	A font where each character has the same width, like Courier New.
	cursive	A font that imitates handwriting, like Comic Sans MS.
font-size	px	Pixels; a fixed unit to define font size, e.g., 16px.
	em	Relative to the font size of the parent element. For example, 1.5em is 1.5 times the parent font size.
	rem	Relative to the root element's font size, e.g., 2rem is 2 times the root element font size.
	%	Percentage; relative to the parent element's font size, e.g., 120% means 120% of the parent's font size.
font-weight	normal	Standard font weight, typically equivalent to 400.
	bold	Bold text, typically equivalent to 700.
	lighter	Lighter font weight than the inherited font.
	100 to 900	Numeric values ranging from thin (100) to very thick (900).
font-style	normal	Normal text style, no slant.
	italic	Italic style, with a slight slant to the text.
	oblique	A more extreme slant than italic.
line-height	1	The line height is equal to the font size.
	1.5	The line height is 1.5 times the font size.
	20px	Fixed height of 20px between lines.
font-variant	small-caps	Text is displayed in small uppercase letters.

2. Text Alignment and Direction

Property	Value	Explanation
text-align	left	Aligns the text to the left of the container.
	right	Aligns the text to the right of the container.
	center	Aligns the text to the center of the container.
	justify	Stretches the text to fit the width of the container, aligning both left and right edges.
direction	ltr	Left-to-right text flow (default for most languages).
	rtl	Right-to-left text flow (used for languages like Arabic or Hebrew).

3. Text Decoration

Property	Value	Explanation
text-decoration	underline	Adds an underline to the text.
	line-through	Adds a strikethrough line across the text.
	overline	Adds a line above the text.
	none	Removes any text decoration.
text-decoration-color	red	Sets the color of the text decoration, such as the underline, to red .
text-decoration-style	solid	The decoration is solid (default).
	dotted	The decoration is dotted (e.g., underlined with dots).
	dashed	The decoration is dashed (e.g., underlined with dashes).
	double	The decoration is double (e.g., two lines beneath the text).
text-decoration-line	underline	Specifies that the decoration is an underline .
	line-through	Specifies a line-through decoration.
	overline	Specifies an overline decoration.

4. Spacing and Indentation

Property	Value	Explanation
letter-spacing	1px	Increases the space between letters by 1px .
	-1px	Decreases the space between letters by 1px .
word-spacing	5px	Increases the space between words by 5px .
	-3px	Decreases the space between words by 3px .
text-indent	30px	Indents the first line of the paragraph by 30px .
	2em	Indents the first line by 2em , relative to the font size.

5. Text Overflow and Wrapping

Property	Value	Explanation
text-overflow	ellipsis	Truncates the text and adds an ellipsis (...) if it overflows the container.
	clip	Clips the text without adding any indication of overflow, cutting off text.
white-space	normal	Collapses whitespace and allows text to wrap to the next line.
	nowrap	Prevents text from wrapping, forcing it to stay on a single line.
	pre	Maintains all whitespace and line breaks as they are, similar to the <pre> HTML tag.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Complete CSS Text Example</title>
  <style>
    /* Element 1: Font Properties */
    .element-1 {
      font-family: "Arial", sans-serif; /* Font Family */
      font-size: 20px; /* Font Size */
      font-weight: bold; /* Font Weight */
      font-style: italic; /* Font Style */
      line-height: 1.8; /* Line Height */
      letter-spacing: 1px; /* Letter Spacing */
      word-spacing: 5px; /* Word Spacing */
      text-transform: uppercase; /* Text Transform */
      color: #008080; /* Text Color */
      background-color: #f0f8ff; /* Background Color */
      padding: 15px;
      margin: 20px 0;
      border: 2px solid #4682b4; /* Border */
    }

    /* Element 2: Text Decoration */
    .element-2 {
      font-family: "Verdana", sans-serif; /* Font Family */
      font-size: 22px; /* Font Size */
      font-weight: normal; /* Font Weight */
      font-style: normal; /* Font Style */
      line-height: 1.6; /* Line Height */
      letter-spacing: 2px; /* Letter Spacing */
      word-spacing: 4px; /* Word Spacing */
      text-transform: capitalize; /* Text Transform */
      text-decoration: underline; /* Text Decoration */
      text-decoration-color: #ff6347; /* Text Decoration Color */
      text-decoration-style: solid; /* Text Decoration Style */
      text-decoration-line: underline; /* Text Decoration Line */
      color: #ff6347; /* Text Color */
      background-color: #e0ffff; /* Background Color */
      padding: 20px;
      margin: 15px 0;
      border: 1px dotted #cd5c5c; /* Border */
    }

    /* Element 3: Text Overflow and Wrapping */
    .element-3 {
      width: 300px; /* Width */
      white-space: nowrap; /* White Space */
      text-overflow: ellipsis; /* Text Overflow */
      overflow: hidden; /* Overflow */
      padding: 20px;
      background-color: #dcdcdc; /* Background Color */
      margin: 15px 0;
      border: 2px solid #a52a2a; /* Border */
    }

    /* Element 4: Text Alignment and Direction */
    .element-4 {
```



```

        text-align: right; /* Text Align */
        direction: rtl; /* Right-to-Left Direction */
        line-height: 2.2; /* Line Height */
        text-indent: 25px; /* Text Indent */
        padding: 20px;
        margin: 15px 0;
        background-color: #ffe4e1; /* Background Color */
        border: 2px dashed #cd5c5c; /* Border */
    }

    /* Element 5: Box Model (Width, Height, Margin, Padding) */
    .element-5 {
        width: 60%; /* Width */
        height: 250px; /* Height */
        padding: 30px; /* Padding */
        margin: 20px 0; /* Margin */
        border: 4px solid #8a2be2; /* Border */
        background-color: #ffb6c1; /* Background Color */
        text-align: center; /* Text Align */
        color: #ffffff; /* Text Color */
    }
</style>
</head>
<body>

    <!-- Element 1: Font Properties -->
    <div class="element-1">
        <p>This is a demonstration of text with Arial font, bold, and italic style, with a
        line height of 1.8. The text is uppercase and spaced out using letter-spacing and word-
        spacing.</p>
    </div>

    <!-- Element 2: Text Decoration -->
    <div class="element-2">
        <p>This text has a capitalized transformation, underline with a red color underline and
        additional letter-spacing. The text is styled with Verdana font and a light cyan background
        color.</p>
    </div>

    <!-- Element 3: Text Overflow and Wrapping -->
    <div class="element-3">
        <p>This is a narrow container where the text is prevented from wrapping and any overflowing text is
        represented by ellipsis. It has a fixed width of 300px, and overflow is hidden.</p>
    </div>

    <!-- Element 4: Text Alignment and Direction -->
    <div class="element-4">
        <p>This text is right-aligned with right-to-left direction, commonly used for languages like
        Arabic. The text-indent is set to 25px, and the line height is adjusted to 2.2 for better
        readability.</p>
    </div>

    <!-- Element 5: Box Model (Width, Height, Margin, Padding) -->
    <div class="element-5">
        <p>This container is 60% of the width of its parent container and has a fixed height of
        250px. It is styled with padding, margin, and a purple border. The text is centered inside with a
        white color.</p>
    </div>

</body>
</html>

```

THIS IS A DEMONSTRATION OF TEXT WITH **ARIAL** FONT, **BOLD**, AND **ITALIC** STYLE, WITH A **LINE HEIGHT** OF 1.8. THE TEXT IS **UPPERCASE** AND SPACED OUT USING **LETTER-SPACING** AND **WORD-SPACING**.

This Text Has A **Capitalized** Transformation, **Underline** With A **Red Color** Underline And Additional **Letter-Spacing**. The Text Is Styled With **Verdana** Font And A **Light Cyan** Background Color.

This is a narrow container where the text is prevented from wrapping.

This text is **right-aligned** with **right-to-left** direction, commonly used for languages like Arabic. The **text-indent** is set to 25px, and the **line height** is adjusted to 2.2 for better readability.

This container is **60%** of the width of its parent container and has a fixed **height** of 250px. It is styled with **padding**, **margin**, and a **purple border**. The text is centered inside with a **white color**.

CSS Styling Links

HTML links are created using the <a> (anchor) tag. By default, links are underlined and blue, but CSS provides many styling options to change their appearance, including colors, text decorations, backgrounds, fonts, and more. You can also style links based on their interaction states (unvisited, visited, hover, and active).

CSS Property	Description	Example
color	Defines the text color of the link.	a { color: red; }
text-decoration	Controls the decoration of text, such as underlining, line-through, or none. Default is underline.	a { text-decoration: none; } (removes underline)
background-color	Sets the background color of the link.	a { background-color: yellow; }
Font Properties	font-family: Specifies the font of the link's text. font-size: Sets the size of the text. font-weight: Defines how bold the text is. font-style: Changes the style (e.g., italic).	a { font-family: Arial; font-size: 18px; font-weight: bold; font-style: italic; }
padding	Adds space inside the link, around the text.	a { padding: 10px 20px; }
margin	Controls the spacing outside the link, separating it from other elements.	a { margin: 5px; }
display	Determines if the link is inline or block-level.	a { display: inline-block; }
cursor	Changes the type of cursor when the user hovers over the link (e.g., pointer, text, etc.).	a { cursor: pointer; }

Link States:

Link State	Description	Example
:link	Applied to an unvisited link .	a:link { color: red; background-color: yellow; }
:visited	Applied to a visited link (after clicking the link).	a:visited { color: green; background-color: cyan; }
:hover	Applied when the user hovers the mouse pointer over the link.	a:hover { color: white; background-color: hotpink; text-decoration: underline; }
:active	Applied when the link is clicked (active state).	a:active { color: blue; background-color: lightblue; }

State Styling Rule Order:

- **:hover** must come after **:link** and **:visited**.

:active must come after **:hover**.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Styled Links</title>
<style>
```

```

/* Unvisited link */
a:link {
    color: red;
    background-color: yellow;
}

/* Visited link */
a:visited {
    color: green;
    background-color: cyan;
}

/* Mouse-over link (Hover) */
a:hover {
    color: white;
    background-color: hotpink;
    text-decoration: underline;
}

/* Active link (clicked) */
a:active {
    color: blue;
    background-color: lightblue;
}

/* General link styling */
a {
    text-decoration: none;
    padding: 10px 20px;
    display: inline-block;
    font-size: 18px;
    font-weight: bold;
    border-radius: 5px;
}
</style>
</head>
<body>

```

Go to Coursera

Go to Udemy

Go to edX

Go to Khan Academy

<h2>Styled Links for Online Education Platforms</h2>

```

<a href="https://www.coursera.org" target="_blank">Go to Coursera</a><br><br>
<a href="https://www.udemy.com" target="_blank">Go to Udemy</a><br><br>
<a href="https://www.edx.org" target="_blank">Go to edX</a><br><br>
<a href="https://www.khanacademy.org" target="_blank">Go to Khan Academy</a>

</body>
</html>

```